

1. Class to be divided into 8 groups with topic allocation: To facilitate doubt clearance and notes/problem related interaction.

2.Sources: Stallings, Hayes, Hamacher, Moris mano, Mathur, Gaonkar (8085)

Note: Based on doubts/Leaves/Holidays/Doubts/Marks the schedule may vary or change

Module-1- Week1-Week5:

Basics: Functional Blocks in a Computer System:

1. Terminology:

Computer Organization vs architecture, program, Von-Neumann architecture Instruction, Data, memory, CPU (MAR, MBR, I/OAR, I/OBR, PC, IR, Execution Unit(ALU) I/O Module, Basic Instruction Cycle (Fetch Cycle + Execution Cycle), Data Transfer (Buses: Processor-Memory, Processor-IO), Control Signal (RD', WR', IO.M'). Instruction Cycle State Diagram, Classes of Interrupts (Program, IO, Timer, HW failure), Flow of Control with Interrupts, Instruction Cycle with Interrupts, Instruction Cycle State Diagram with interrupts.

2. Teaching Pedagogy: Diagrams and 8085 simulator

- Program → Instruction → Memory Storage with a Diagram → Converted into an ALP → with Address locations and address space(0000 H- FFFF H) Address Bus → PC and MAR → CPU → Fetch address Data in Data Bus in MBR → Four Basic Operations(MR,MW,IOR,IOW) → Fetch cycle → Instruction decode(8085 Reference 8 bit) → Data Fetch → ALU operations-Example(With Instruction format) → Describing the state Diagram using the above example.
- Program → Instruction → Memory → Classes of Interrupts → interrupt like function calling in C Language(Program), IO(As address in main memory, IO-Mapped IO and Memory Mapped IO) → With and Without with commands → Advantages of using Interrupt cycle to improve the efficiency in CPU Utilization, Types of interrupts(short and Long), State Diagram with interrupts.

Expected Learning outcomes:

The student able to visualize

1. How a program is stored?
2. How the instructions are accessed?
3. How the components of CPU work together to execute a given instruction in Assembly language program using opcode and operands (Instruction Format)?
4. How and what type of interrupts can be handled and also improve CPU utilization.

Number system: Covered during First Year

Computer Arithmetic:

1. **Terminology:**
ALU, Integer Representation, Sign Magnitude, 1's and 2's complement, Range, Fixed point Presentation, Negative numbers representation, Addition Subtraction in 2's complement, Overflow Rule,
2. **Teaching Pedagogy:**
Examples and Exercises
3. **Expected Learning outcomes:**
 - The students should be able to represent numbers in 2's complement form and also do the necessary operations.

Gates:

1. **Terminology:** AND, OR, NOT, NAND, NOR, XOR, XNOR, Truth Tables, symbols, Diagrams
2. **Teaching Pedagogy:**
Memory Based (Drawing + Truth Tables)
3. **Expected Learning outcomes:**
The student should be able to draw the truth tables and design the circuits.

Boolean algebra:

1. **Terminology:**
Signal, Boolean Literals, Boolean Laws, Truth Tables, Truth Tables to Boolean Function SOP, POS, CANONICAL FORMS AND INTER CONVERSIONS, Maxterms, Minterms, KARNAUGHMAP: 2,3, VARIABLE MAP (Overlapping, Singleton, Circular Map, Largest Group, Redundant Group)
2. **Teaching Pedagogy:**
Examples and Exercises
3. **Expected Learning outcomes:**
The student should appreciate and understand the utility of Boolean Algebra to design a simplified Circuit using Less number of Gates

Combinational logic Circuits:

1. **Terminology:**
Combinational Circuits, Half Adder, Full-adder, Multibit -Adder, Multiplexer, Decoder
2. **Teaching Pedagogy:**
General Methodology to design a combinational circuit → Half Adder Circuit → Full -Adder Multiplexer Circuit design using the steps → application (Notations used for Counter, Branching and Displacement addressing mode to generate PC for next instruction) Decoder Circuit → application (Chip Select using few bits for the address bus)
3. **Expected Learning outcomes:**
Students should be able to design the discussed circuits with few variations of 8:1 MUX and 3x8 decoder etc.

Sequential Logic circuits:

1. **Terminology:**
Sequential Circuits, Flip-Flops (SR, D, JK,T), Truth Table, Characteristics table and excitation Table, Counter(Synchronous and Asynchronous), and shift Register.
2. **Teaching Pedagogy:**
General Methodology for understanding all basic flip-flops, Methodology to design counters and Shift Registers
3. **Expected Learning outcomes:**
Students should be able to derive the three tables for each kind of flip flop and design the desired circuits.

Module-2: Week 6 – Week 8

Addressing Modes:

1. **Terminology:**
Registers in CPU, Immediate, Direct, Indirect, Register, Register Indirect, Displacement Address, Stack Addressing Mode
2. **Teaching Pedagogy:**
Using the 8085 simulator and simple programs to explain the modes.
3. **Expected Learning outcomes:**
Students should be able to explain the addressing modes and examples from 8085

Instruction Set:

1. **Terminology:**
Instruction Format, Elements of Instructions, Operand types, Instruction Cycle, n—Address Instructions (n=1,2,3), Instruction Types, Instruction Set Design.
2. **Teaching Pedagogy:**
8085 simulators will be used to describe the instructions set, types, and memory occupation (LSBs and MSBs) →operations using the instructions→ Operand types (Data, address, Register) →code types and sizes (MOV, MVI LDA LXI etc.)
3. **Expected Learning outcomes:**
Students will have a fair idea about instruction types, instructions, and their mode of usage

Processor Function:

1. **Terminology:**
2. **Teaching Pedagogy:**
3. **Expected Learning outcomes:**